

Bases de Datos Activas

Introducción

BD Pasivas vs. Activas

Bases de datos pasivas

- Son meros almacenes de datos.
- Toda a xestión é explícita.

Bases de datos activas

- Actividade básica: controles mínimos con restricións (clave primaria, foránea, ...).
- Máis avanzada: A BD "Reacciona" ante eventos (normalmente cambios nos datos)
- Utilidade (exemplos):
 - Control de restricións
 - Reglas de negocio: Un empleado non pode gañar máis que o seu xefe.
 - Validacións: NIF correcto, salario positivo, sexo é H ou M, ...
 - Actualización de datos
 - Xeneración de claves primarias
 - Tabla facturas con campo total autoactualizado
 - Accións externas á base de datos
 - Alerta de stock mínimo de produtos
- A actividade máis avanzada implméntase normalmente con trigger- A actividade máis avanzada implméntase normalmente con trigger- A actividade máis avanzada implméntase normalmente con triggers

Actividades sen triggers

Restriccións

- NOT NULL
- UNIQUE e clave primaria
- CHECK
- Clave foránea

Consideracións

- A acción sempre é abortar (excepto accións referenciais en FK)

- **check**: só a nivel de fila.
- Non reacciona ante borrado de filas
- Ventaxa: Poden aplazarse ata o fin da transacción

```
CREATE TABLE EMP(
    EMPNO NUMERIC(3) NOT NULL,
    ENAME CHAR(20) NOT NULL,
    EMAIL CHAR(25) NOT NULL CONSTRAINT U_EMAIL UNIQUE,
    MGR NUMERIC(3), DEPTNO NUMERIC(3),
    SAL NUMERIC(7,2),
    CONSTRAINT PK_EMP PRIMARY KEY(EMPNO),
    CONSTRAINT FK_EMP FOREIGN KEY(MGR)
        REFERENCES EMP(EMPNO) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT C_SAL_POS CHECK(SAL > 0) DEFERRABLE INITIALLY DEFERRED);
```

Restriccións a nivel de BD: Assertions

```
CREATE ASSERTION <nome> CHECK <condición>
```

```
CREATE ASSERTION as_salarios_similares
    CHECK( ( (SELECT MAX(SAL) FROM EMP) -
            (SELECT MIN(SAL) FROM EMP) ) <100);

CREATE ASSERTION deptnos_pequenos
    CHECK (NOT EXISTS
        (SELECT DEPTNO FROM EMP
         GROUP BY DEPTNO
         HAVING COUNT(*) > 10));
```

Limitacións

- A reacción sempre é abortar
- Os SXBD non as implementan

Triggers

Triggers: Programación baseada en eventos.

Tamén chamados regras ECA (Evento-Condición-Acción)

- Cando se produce o evento, o trigger *dispárase*.
- A condición é opcional (se non existe, suponse que é certa).
- Se a condición é certa, execútase a acción.

Exemplos de uso:

1. Control de salario: o salario debe ser positivo.
2. Control de NIF inválido.
3. Un departamento non pode ter máis de 10 empregados.
4. Un emprego non pode ganar máis ca o seu xefe.
5. Creación de claves subrogadas (parecido a un campo autoincremental).
6. Actualiza o total dunha factura cando se engaden/borran/modifican liñas.
7. Gravar log de todos os accesos que modifican datos na táboa de empregados.
8. Manda un correo ó suministrador cando o stock dun artigo baixa do límite.

Definición de regras ECA para algún dos exemplos previos

A este nivel especificamos que facer, non como o faremos.

- (1) Control de salario inválido
 - E: insert; C: salario <= 0; A: Non permitir a inserción
 - E: update; C: salario <= 0; A: Non permitir a actualización
- (5) Creación de clave surrogada (campo id)
 - E: insert; C: -; A: Xerar novo id
- (7) Log de modificación na táboa de empregados
 - E:insert; C: -; A: Gravar entrada no log
 - E:delete; C: -; A: Gravar entrada no log
 - E:update; C: -; A: Gravar entrada no log

A proposta do estándar SQL

Triggers no estándar SQL

- Dentro do estándar ISO/IEC 9075 Part 2: (SQL/Foundation)
- Especificados no SQL:1999 (antes SQL-3)
- Algunhas ampliacións en SQL:2003-SQL:2008
- Un trigger "controla" unha única táboa (ou vista)
- Eventos: insert, delete, update.
- Un trigger responde a un único evento.
- Un trigger non valida os datos existentes (como fan as restricións salvo se usamos novalidate)

Ventaxas dos triggers

- Simplifican a codificación de aplicacións de acceso a BDs
- Maior consistencia (control centralizado)

Triggers en SQL:2008

Elementos dun trigger

- Nome do trigger
- Nome da táboa
- Evento: insert, delete, update [OF <lista_columnas>]
- Tempo de activación:
 - Antes (`before`) ou despois (`after`) do evento.
 - En vez de (`instead of`): *Non core, "Feature T213". Só aplicable a vistas.*
- Granularidade: `for each {row | statement}`

Cantas veces se dispara o trigger para un evento: *Implica distintas versións dos datos: táboas e variables de transición*

 - A nivel sentencia (`statement`): unha vez para todo o evento.
 - A nivel fila (`row`): unha vez para cada fila afectada.
- Condición: `when (<condición>)`. Pode omitirse
- Acción:
 - Debe ser atómica.
 - O seguinte non se recomenda (aberto a implementación):
 - DML de modificación de datos en triggers `before`. (Podemos usar asignacións a variables de transición con `set`).
 - Sentencias de control de transaccións, conexión a BD, ou DDL.

Elementos dun trigger (ii)

Táboa e variables de transacción

- Valores antigos (`old`) e novos (`new`) visibles para o trigger.
- A veces chamados "nomes de correlación".
- Se son usados, é obrigatorio decláralos na cláusula referencing.
- Variables de transacción: para `triggers for each row`
 - Valores da fila previo e posterior á execución do evento.
 - Nomenclatura: `old [row]`, `new [row]`
 - Non ambos valores son aplicables en todos os eventos (ex: `old row` para insert)
- Táboa de transición para `triggers for each statement`
 - Valores previo e posterior das filas afectadas polo evento.
 - Nomeclatura: `old table`, `new table`.
 - Non se refire á táboa completa (nomenclatura enganosa)!, só ó conxunto de filas afectadas.

- Non completamente implementado en SXBD reais. (o mais habitual é que vexan só a táboa completa antes/despois do evento, dependendo do tempo de activación).

// Meter foto de la diapositiva 9

Exemplos

Exemplo 1: Control de salario

- Usamos a táboa de exemplo de oracle:

```
emp(empno, ename, job, mgr, hiredate, sal, comm, deptno)
```

- Podería resolverse cunha restrición ou cunha aserción:

```
ALTER TABLE emp
  ADD CONSTRAINT sal_pos CHECK (sal>0)

CREATE ASSERTION emp_sal_pos
  CHECK (NOT EXISTS (SELECT * FROM emp WHERE sal <=0))
```

- Comentarios:
 - Funcionan tanto para insert como para update.
 - Comproban os datos existentes en BD.
 - Poderían facerse aplazables ata o fin da transacción.
 - Provocan erro e a inserción ou actualización non se leva a cabo.

Usando triggers:

- Para a inserción, como podemos eliminar o dato insertado (de forma transparente ó usuario), ou xenerar unha excepción.

```
CREATE TRIGGER t_emp_salpos_air
  AFTER INSERT ON emp          -- Debe ser AFTER
  REFERENCING NEW ROW AS nova
  FOR EACH ROW
  WHEN (nova.sal <=0)
  DELETE FROM emp
    WHERE empno = nova.empno;

CREATE TRIGGER t_emp_salpos_bir
  BEFORE INSERT ON emp        -- Serviría BEFORE ou AFTER
  REFERENCING NEW ROW AS nvoa
  FOR EACH ROW
```

```
WHEN (nova.sal <= 0)
  SIGNAL SQLSTATE '99001', 'Salario inválido';
```

Para a actualización, deixando o salario anterior: Facer un novo update?

```
CREATE TRIGGER t_emp_salpos_aur
  AFTER UPDATE OF sal ON Emp          -- Debe ser AFTER
  REFERENCING NEW ROW AS nova
    OLD ROW AS vella
  FOR EACH ROW
  WHEN (nova.sal <= 0)
  UPDATE Emp SET sal = vella.sal      -- Dispara o trigger de novo
  WHERE empno = nova.empno;          -- Menos eficiente
```

Mellor cambiar o valor na fila actual utilizando set (só válido nos triggers before).

```
CREATE TRIGGER t_emp_salpos_bur
  BEFORE UPDATE OF sal ON Emp         -- Debe ser BEFORE
  REFERENCING NEW ROW AS nova
    OLD ROW AS vella
  FOR EACH ROW
  WHEN (nova.sal <= 0)                -- "Intercepta" o valor e
  substitúeo                          -- polo anterior antes de gravar
  SET nova.sal = vella.sal;
```

Exemplo 5: Xeneración de id subrogado

- Usamos unha secuencia
- Creamos un trigger que, no caso da inserción substitúe o valor do identificador polo próximo calor da secuencia.
- Ó modificar (con `set`) o valor nova, debe ser de tipo before.

```
-- Táboa: t1(id primary key, ...)

CREATE SEQUENCE seq_t1;

CREATE TRIGGER t_t1_id_bir
  BEFORE INSERT ON t1                -- Debe ser BEFORE
  REFERENCING NEW ROW AS nova
  FOR EACH ROW
  SET nova.id = NEXT VALUE FOR seq_t1;
```

Exemplo 7: Log de modificacións

- Trigger a nivel de sentencia: rexistra cada sentencia de modificación sobre `EMP`.
- Usamos unha táboa `log(id, usuario, evento, timestamp)`
- Usaremos unha secuencia `seq_log` para o identificador.
- Poderíamos incluso almacenar no log os valores vellos/novos das modificacións (requeriría trigger a nivel de fila).
- Como non afecta á táboa, pode ser `before` ou `after`.

```
-- Móstrase só o trigger para insert.
-- Para delete e update serían análogos.

CREATE TRIGGER t_emp_log_aix
  AFTER INSERT ON EMP                                -- Serviría BEFORE ou AFTER
  FOR EACH STATEMENT
  INSERT INTO log(id, usuario, evento, timestamp)
    VALUES(NEXT VALUE FOR seq_log,
            CURRENT_USER,
            'insert',
            CURRENT_TIMESTAMP);
```

Triggers INSTEAD OF

- Triggers utilizados só con vistas (non táboas).
- Non se permite a condición (cláusula when).
- Execútase a acción do trigger en vez de executar o evento.
- Utilidade principal: Facer actualizable unha vista que non o é.

```
create view vempl as
  select deptno, count(*) as numemps
  from emp
  group by deptno;

delete from vempl                                -- Falla!! Vista non actualizable.
  where deptno=10;

CREATE TRIGGER t_vempl_borrado_iodr
  INSTEAD OF DELETE ON vempl
  REFERENCING OLD ROW AS vella
  FOR EACH ROW
  DELETE FROM emp WHERE deptno = vella.deptno;

delete from vempl                                -- Funciona! Borra todos os empregados do
```

```
depto 10.  
  where deptno=10;
```